

Projeto de redes neurais artificiais utilizando um MLP (Multilayer Perceptron) para classificação de sobreviventes do Titanic

Arthur Augusto Eich, Rafael Gotz, Joao Vitor

Centro de Ciências Exatas e Tecnológicas – Universidade Estadual do Oeste do Paraná (UNIOESTE)

R. Universitária, 1619 - Universitário, Cascavel - PR, 85819-110

{arthur.eich, rafael.gotz, joao???}@unioeste.br

Abstract. *This paper describes a classification process with the purpose of predicting the survival - or not - of a certain person throughout the shipwreck of the transatlantic ship Titanic in 1912 using Multilayer Perceptron, a class of artificial neural networks.*

Resumo. *Este artigo descreve um processo de classificação com o objetivo de prever a sobrevivência - ou não - de uma determinada pessoa ao naufrágio do navio transatlântico Titanic em 1912 utilizando Multilayer Perceptron, uma classe de redes neurais artificiais.*

1. Introdução

As redes neurais são uma classe de modelos de aprendizado de máquina inspirados no funcionamento do cérebro humano. Elas são compostas por unidades interconectadas chamadas de neurônios artificiais, que são organizados em camadas. Cada camada recebe entradas, processa as informações e passa os resultados para a próxima camada até chegar à camada de saída. Neste processo, cada neurônio em uma camada recebe as saídas dos neurônios da camada anterior, realiza um cálculo ponderado dessas entradas e passa o resultado por uma função de ativação para gerar a saída. As funções de ativação introduzem não-linearidades no modelo, permitindo que ele aprenda relações complexas nos dados.

Neste projeto, iremos fazer uso de redes neurais para classificar a sobrevivência de um passageiro ao naufrágio do Titanic, com base em suas características, condições financeiras, entre outros.

2. Problema

O problema consiste em um processo de classificação binária acerca da possibilidade ou não de sobrevivência de uma pessoa arbitrária ao naufrágio do Titanic, em 1912. A resolução, através de MLP, utilizando a biblioteca Keras para Python e segue as etapas essenciais à criação de um modelo desse tipo:

- Especificação da arquitetura;
- Compilação;
- Ajuste;
- Predição;

2.1. Arquitetura utilizada

É uma rede neural com três camadas: uma camada de entrada, uma camada oculta e uma camada de saída:

- A camada de entrada tem 64 neurônios.
- A camada oculta tem 32 neurônios com função de ativação ReLU.
- A camada de saída tem 2 neurônios, correspondendo às duas classes de saída possíveis - sobreviveu ou não sobreviveu. É utilizada a função de ativação Softmax nesta camada para gerar uma distribuição de probabilidade sobre as classes.

Esta arquitetura é comumente usada para resolver problemas de classificação binária ou multiclasse.

Foi utilizado o otimizador Adam, sendo uma boa escolha para problemas de classificação, pois é rápido e eficiente, bem como a função de perda cruzada categórica é uma escolha interessante para problemas de classificação, dado que é uma medida precisa do erro entre as saídas do modelo e os rótulos de treinamento.

3. Métodos Utilizados

3.1. Multilayer Perceptron (MLP)

A MLP é uma classe de redes neurais artificiais que consiste em várias camadas de neurônios interconectados. Cada neurônio recebe entradas, processa essas informações e passa os resultados para a próxima camada até chegar à camada de saída. A MLP é amplamente utilizada em problemas de classificação e regressão.

3.2. Validação Cruzada

A Validação Cruzada é uma técnica importante para avaliar o desempenho de modelos de aprendizado de máquina. Nesse projeto, foi utilizada a técnica de Validação Cruzada com k-Folds, onde o conjunto de treinamento é dividido em k partes (ou "folds"). O modelo é treinado k vezes, cada vez utilizando k-1 folds como conjunto de treinamento e o fold restante como conjunto de validação. Isso permite uma avaliação

mais robusta do desempenho do modelo em diferentes subconjuntos dos dados de treinamento

4. Algoritmo de treinamento utilizado

O algoritmo de treinamento utilizado foi o Gradiente Descendente Estocástico (SGD), uma técnica de otimização muito comum para redes neurais. O SGD atualiza os pesos da rede iterativamente, utilizando pequenos lotes aleatórios de dados para calcular os gradientes.

O modelo foi compilado com o otimizador Adam e a função de perda categórica de entropia cruzada, adequada para problemas de classificação. Durante o treinamento, foram utilizados callbacks para salvar o melhor modelo obtido em cada fold da Validação Cruzada, evitando overfitting e permitindo a utilização do melhor modelo para avaliação final.

Para resolver o problema de classificação da sobrevivência no Titanic, utilizamos uma rede neural do tipo feedforward, a arquitetura mais comum de redes neurais. A rede possui três camadas: a camada de entrada, uma camada oculta e a camada de saída.

5. Pré-processamento dos Dados

Antes do treinamento, os dados foram pré-processados para garantir que os neurônios nas camadas ocultas não dominassem o processo de aprendizado. Foi realizada a normalização dos dados, que consiste em subtrair a média dos dados e dividir pelo desvio padrão. Esse processo é importante para evitar que características com escalas muito diferentes tenham um impacto desproporcional no modelo.

6. Parâmetros utilizados na solução desenvolvida

- Camada de Entrada e variáveis de entrada:

A camada de entrada possui 11 neurônios, correspondendo às 11 variáveis de entrada - survived (sobreviveu), pclass (classe), age (idade), sibsp (siblings / partners - quantidade de irmãos / cônjuges a bordo do Titanic), parch (quantidade de pais / crianças a bordo do Titanic), fare (tarifa paga), male (homem), age_was_missing (idade faltando), embarked_from_cherbourg (embarcou de cherbourg), embarked_from_queenstown (embarcou de queenstown) e embarked_from_southampton (embarcou de southampton).

- Camada Oculta:

Optamos por uma única camada oculta com 8 neurônios. A escolha do número de neurônios em camadas ocultas é um hiperparâmetro que pode ser ajustado para obter o melhor desempenho do modelo.

- Camada de Saída e variáveis de saída:

A camada de saída possui 2 neurônios, correspondendo às duas classes de saída -

sobreviveu ou não sobreviveu. A saída da rede, correspondente a se o passageiro sobreviveu ou não sobreviveu, foi codificada utilizando a codificação one-hot, onde a classe de saída é representada como um vetor binário com 1 na posição correspondente à classe e 0 nas demais posições.

- **Função de Ativação:**

Utilizamos a função de ativação ReLU (Rectified Linear Unit) para os neurônios das camadas ocultas, e a função de ativação Softmax para os neurônios da camada de saída. A função ReLU introduz não-linearidade na rede, permitindo que ela aprenda relações complexas nos dados. A função Softmax é aplicada na camada de saída para gerar uma distribuição de probabilidade sobre as classes.

- **Taxa de Aprendizado:**

A taxa de aprendizado foi empregada durante o treinamento do MLP para classificação de sobreviventes. Ela afetou a eficiência do algoritmo de otimização usando para ajuste de pesos da rede, como o algoritmo SGD utilizado. Um valor muito baixo pode levar a uma convergência lenta, enquanto um valor muito alto pode fazer com que o algoritmo oscile em torno da solução ótima sem alcançá-la.

No processo de busca de hiperparâmetros utilizando o GridSearch, foi testado diferentes valores para a taxa de aprendizado e observado como ela afetou o desempenho do modelo. Encontrar a taxa de aprendizado ideal é de grande importância para garantir que o modelo aprenda de maneira eficiente e alcance uma boa precisão na classificação dos sobreviventes.

Além disso, o otimizador Adam foi ajustando-a automaticamente com base nas estimativas do momento do gradiente e do momento do gradiente ao quadrado

- **Ajuste do modelo:**

Consiste em aplicar retropropagação e descida de gradiente com seus dados para atualizar os pesos, escalando tais dados antes de ajustar o modelo ajuda a garantir que o modelo aprenda de maneira eficiente.

Para MLPs, é importante escalar os dados de maneira uniforme, pois isso ajuda a evitar que os neurônios nas camadas ocultas dominem o processo de aprendizagem. Uma maneira comum de escalar os dados é normalizá-los, o que significa subtrair a média dos dados e dividi-los pelo desvio padrão.

Depois de ajustar o modelo, seu desempenho deve ser validado e avaliado em um conjunto de dados de teste para garantir que ele não esteja apenas aprendendo os dados de treinamento.

7. Implementação

A implementação se deu utilizando a ferramenta de programação Python e consiste em um conjunto de arquivos que configuram:

- O carregamento dos dados (data_loading.py)
- A predição sobre o conjunto de testes (make_predictions.py)

- O desenvolvimento e controle do modelo (model_evaluate.py)
- A configuração geral do modelo, incluindo sua compilação (model_setup.py)
- O treinamento do modelo (model_train.py)

Os módulos/funções e bibliotecas para Python empregados no projeto são os seguintes:

- Num Py, completo
- Pandas, completo
- Matplotlib, completo
- To_categorical, da biblioteca Keras/TensorFlow
- Train_test_split, da biblioteca Sklearn
- Dense, da biblioteca Keras/TensorFlow
- Sequential, da biblioteca Keras/TensorFlow
- StratifiedKFold, da biblioteca Sklearn
- Plt, da biblioteca Matplotlib
- ModelCheckpoint, EarlyStopping, da biblioteca Keras

8. Resultados

Os resultados são, em última instância, salvos em um arquivo do tipo .csv (comma-separated values), de coluna única e contendo as probabilidades encontradas pelo modelo treinado para o conjunto de testes utilizado. A acurácia média do modelo foi calculada utilizando a Validação Cruzada com k-Folds. Esse valor representa o desempenho geral do modelo em diferentes subconjuntos dos dados de treinamento. Quanto maior a acurácia média, melhor o desempenho do modelo.

O modelo desenvolvido apresentou uma taxa de acerto satisfatória na tarefa de classificação dos sobreviventes do Titanic.

Imagens

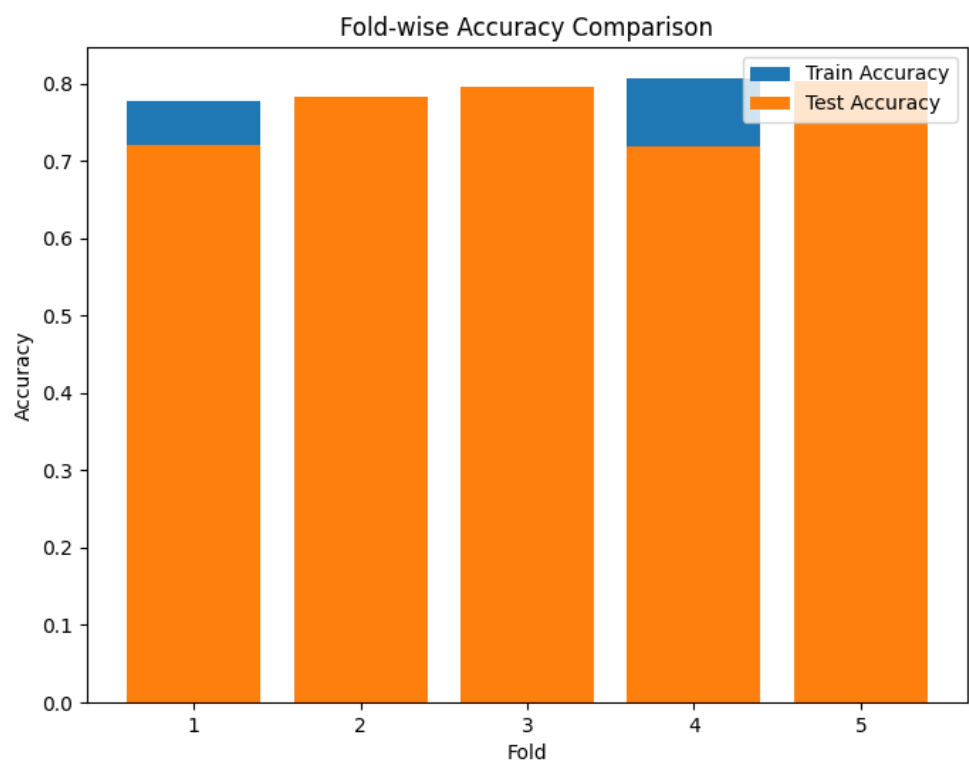


Figura 1. Acurácia do conjunto de treinamento x testes por dobras

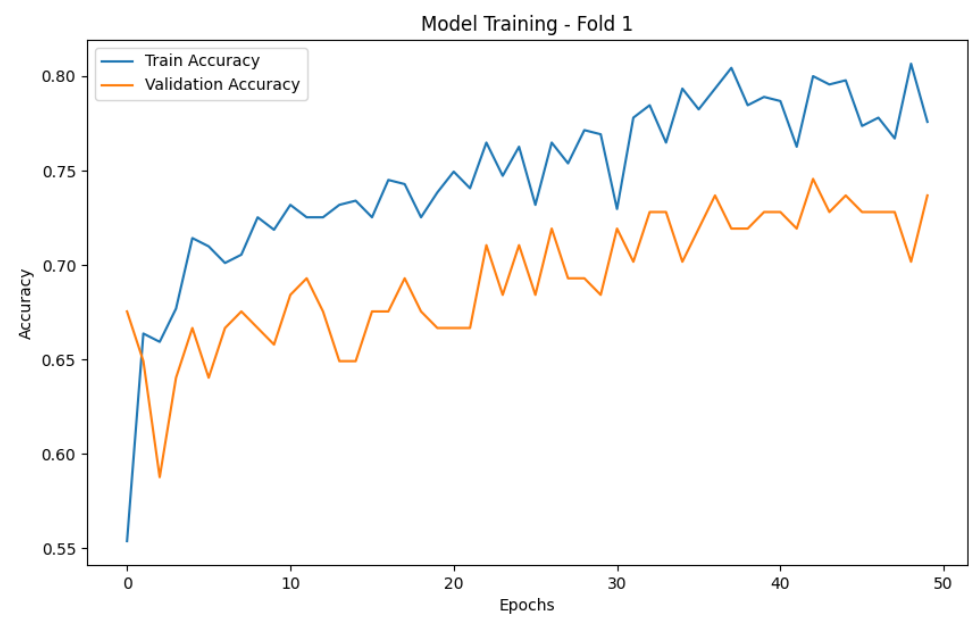


Figure 2. Dobra nº 1

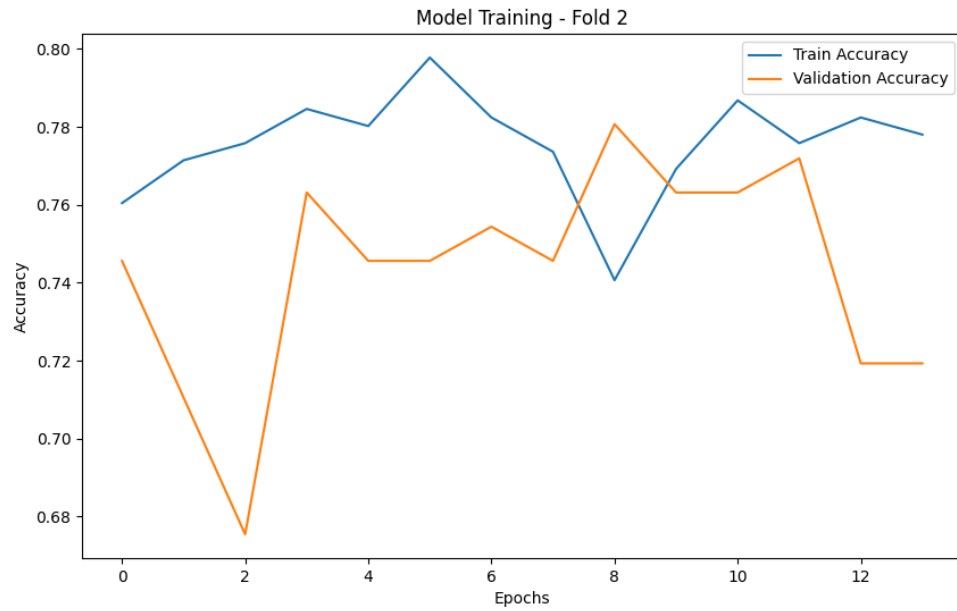


Figure 3. Dobra n° 2

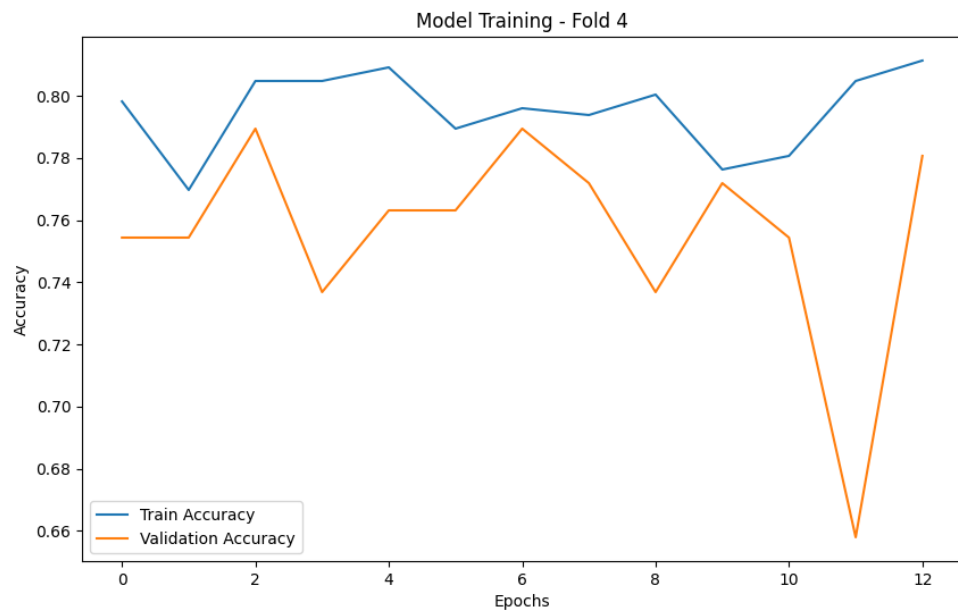


Figura 4. Dobra n°3

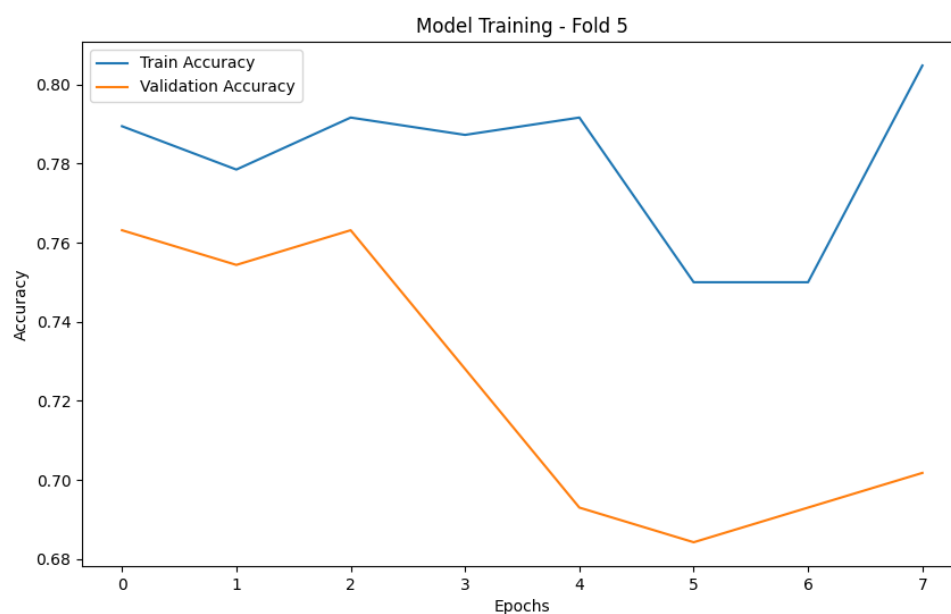


Figure 5. Dobra nº 4

Tabela 1. Predição de um conjunto de testes com 10 observações

10.96%

100.00%

2.79%

14.89%

3.05%

7.87%

25.18%

51.99%

3.82%

19.17%

9. Conclusão

Este trabalho explora o uso de redes neurais artificiais do tipo Multilayer Perceptron para classificar os sobreviventes do naufrágio do Titanic. Por meio de um processo de pré-processamento de dados e ajuste de hiperparâmetros, desenvolvemos um modelo que pode prever com precisão a sobrevivência do passageiro com base nas características do passageiro.

Os resultados obtidos são promissores e demonstram que a abordagem de redes neurais é eficaz na resolução de problemas de classificação tão complexos. Pesquisas futuras podem se concentrar em explorar outras arquiteturas de redes neurais e incorporar mais recursos para melhorar ainda mais o desempenho do modelo.

10. Referências

- GAD, Ahmed Fawzy; JARMOUNI, Fatima. **Introduction to Deep Learning and Neural Networks with Python: A Practical Guide**. 1. ed. [S. l.]: Academic Press, 2020.
- LOY, James. **Neural Network Projects with Python**: The ultimate guide to using Python to explore the true power of neural networks through six projects. 1. ed. [S. l.]: Packt Publishing, 2019.